LR-Agent / Agent 后端入门

第07章
模型适配与推理工具接入

从模型构造、视觉附图到本地bbox结果，读懂Agent如何接入能力

读者起点与本章边界

前置：已完成第 1、2 章基础实验。只围绕后端输入、决策、执行与验证学习；遇到新名词先看定义和小例子。真实截图、源码阅读图与概念示意分别注明。

学完本章应该能够

- 区分主模型、辅助模型、视觉模型能力与本地检测模型。
- 按真实源码解释模型配置、附图时机和推理输入输出。
- 运行3个无网络实验，准确解释归一化坐标与待确认结果。
- 把功能验收与模型质量分开，形成可被面试追问的证据。

源码核对：2026-10-17，工作区基线 d5dd009；本轮包含已验证的日志断管道修复。正式环境 LangChain 1.4.2 / LangGraph 1.2.12。

本章目录与实验准备

7.1 从一句标注需求拆出三种能力 . . . 3
7.2 名词卡：模型、接口与工具不是一个对象 . . . 4
7.3 源码入口：build_llm_pair只负责构造 . . . 5
7.4 完整实验：用假工厂观察模型配置 . . . 6
7.5 主模型与辅助模型：按职责选择 . . . 7
7.6 视觉输入：路径字符串并不是图像 . . . 8
7.7 视觉fallback：首轮漏调工具才兜底 . . . 9
7.8 本地inference：另一套输入输出协议 . . . 10
7.9 bbox名词：先明确坐标约定 . . . 11
7.10 完整实验：手算并校验一个bbox . . . 12
7.11 从推理结果到提案：完成不等于保存 . . . 13
7.12 完整实验：验证结果状态的独立含义 . . . 14
7.13 真实验收图：把截图当作证据的一部分 . . . 15
7.14 功能正确、模型正确、工作更快：分开验证 . . . 16
7.15 练习：换模型、换图片时先查什么 . . . 17
7.16 面试复盘：接入能力的边界要说清 . . . 18
7.17 图解：一行JSON怎样表达一次推理 . . . 19
7.18 源码图解：模型配置怎样进入实例 . . . 20

每个实验包含全部导入、输入与执行。固定模型和教学数据只验证机制；源码节选只用于阅读。请先预测输出，再运行并改变一个输入。

```
Set-Location D:\Study\LR-Agent
$env:PYTHONPATH = Join-Path (Get-Location) 'LR-Agent\vendor\local-agent'
$env:LANGSMITH_TRACING = 'false'
$env:LANGCHAIN_TRACING_V2 = 'false'
$agentPy = 'D:\Study\LR-Agent\.venvs\langchain-main\Scripts\python.exe'
```

7.1 从一句标注需求拆出三种能力

场景：用户说“给 data/1.jpg 自动标注”。如果只把图片路径交给聊天模型，它并不会自然变成磁盘上的标注文件。后端需要理解请求、选择工具、等待执行结果，再解释结果。检测模型则负责从图像预测实例；两类模型分工不同。



图7-1 概念分工：本章关注Agent后端集成

参与者	负责什么	不由它直接保证
主语言模型	对话、规划下一步调用	检测精度与磁盘写入
辅助模型	只读子代理的查阅总结	自动多模型投票
检测模型	输出类别、置信度和几何	用户确认与业务幂等

学习边界

只学后端如何接入这些能力，不展开桌面UI、模型训练或RAG。外部执行方作为协议参与者出现即可。

7.2 名词卡：模型、接口与工具不是一个对象

Model（模型）是一组能够处理输入并产生输出的计算能力；Provider（服务提供方）负责托管接口；Adapter（适配器）把某种接口转换成框架可调用的统一对象。LangChain帮我们统一调用方式，不替模型完成训练，也不替服务方保证能力。

术语	通俗理解	在项目里看哪里
ChatOpenAI	兼容某类聊天接口的适配对象	entry.py::build_llm_pair
Tool calling	输出结构化调用请求	AIMessage.tool_calls
Vision	能读取图像内容的模型能力	supports_vision与附图消息
Inference	已训练模型对新输入做预测	vendor/inference/runners

同一个模型名在不同服务端可能支持不同参数，能回答文本不等于支持工具调用。通过视觉探针也不等于检测框质量达标。先检查服务能力，再检查任务结果，最后检查业务副作用。

先问输入输出

“用了什么模型”之后还应回答：输入是消息还是像素？输出是工具请求、文字，还是带坐标的实例？这比背API名称更接近真实工程。

7.3 源码入口：build_llm_pair只负责构造

函数名中的 build 表示构造，pair 表示一对；body 是通过Pydantic校验的 LocalChatStreamRequest。返回 tuple[Any, Any] 表示两个返回值：主模型对象和辅助模型对象。辅助配置不完整时第二项为 None。Any说明该层允许替换模型实现，并不等于所有模型都兼容。

源码节选：LR-Agent/vendor/local-agent/app/agent/langchain_runtime/entry.py::build_llm_pair（主模型构造段）

```
base_url = body.base_url.rstrip("/")
llm = factory(
    model=body.model,
    api_key=body.api_key,
    base_url=base_url,
    streaming=True,
    temperature=0.7,
    timeout=120,
    extra_body=_deepseek_extra_body(body.model, base_url),
)
```

参数/函数	含义	阅读提醒
rstrip("/")	去除URL尾部斜线	不是网络连通性检查
streaming=True	请求流式输出	还需模型端支持
timeout=120	传给适配器的超时配置	不是业务总耗时保证
extra_body	接口额外请求参数	DeepSeek分支关闭thinking

源码已确认

构造适配对象不等于已经发起请求；真实调用发生在 invoke/astream 等执行阶段。本节源码是节选，未展示辅助模型分支。

7.4 完整实验：用假工厂观察模型配置

想检查模型配置是否传对，不必先配置云服务。用一个只记录参数的函数替换工厂，运行真正的 build_llm_pair，就能观察构造行为。此方法适用于配置映射、URL处理和主辅选择的离线测试。

完整教学实验 ch07_factory。在 study 自行新建 my_ch07_factory.py，复制全部代码。

```python
from app.agent.langchain_runtime import entry
from app.schemas.agent import LocalChatStreamRequest

calls = []
def fake_factory(**config):
    calls.append(config)
    return {'model': config['model']}

body = LocalChatStreamRequest(
    api_key='teaching-placeholder',
    base_url='https://example.invalid/v1/', model='main-demo',
    aux_api_key='teaching-placeholder', aux_model='aux-demo',
    aux_base_url='https://example.invalid/v1/',
    messages=[], user_content='explain tools', client_job_id='demo-7',
)
original = entry.ChatOpenAI
try:
    entry.ChatOpenAI = fake_factory
    main, aux = entry.build_llm_pair(body)
finally:
    entry.ChatOpenAI = original
assert len(calls) == 2 and calls[0]['streaming'] is True
assert calls[0]['timeout'] == 120
print('main:', main['model'])
print('aux:', aux['model'])
print('base_url:', calls[0]['base_url'])
print('network_calls: 0')
```

运行：& $agentPy ".\study\my_ch07_factory.py"（工作区根目录）

本轮实际输出：

```
main: main-demo
aux: aux-demo
base_url: https://example.invalid/v1
network_calls: 0
```

工作目录为项目根目录；先按本章运行约定设置 PYTHONPATH，再用正式解释器运行 my_ch07_factory.py。占位配置不会读取个人模型设置。

**config 把传入的具名参数收进字典；fake_factory 只记录配置并返回字典，完全没有网络客户端。finally 无论成功或异常都会恢复被替换的工厂。

输出证明两个模型分开构造、URL尾部斜线被去掉、流式与超时配置已传入；没有证明真实服务支持工具调用，也没有测延迟。

7.5 主模型与辅助模型：按职责选择

当前入口把 llm 交给主Agent。只读 explore_readonly 子代理使用 aux_llm or llm：如果有辅助模型就选辅助模型，否则退回主模型。Python的 or 在这里返回第一个真值对象，不能把它理解为同时调用两个模型。



图7-2 实际装配关系：不是双模型投票

选择	可能收益	需要额外验证
只用主模型	配置少、行为较统一	查阅子任务成本
独立辅助模型	按子任务选成本和能力	工具兼容、摘要质量
盲目增加模型	没有必然收益	延迟、失败点和调试负担

设计取舍

源码证明了职责路由；尚未测量主辅组合的费用下降或速度提升。面试可解释选择逻辑，不能直接声称成本降低某个百分比。

7.6 视觉输入：路径字符串并不是图像

给模型一句“看看 data/1.jpg”只提供了文本路径。远端模型无法直接访问本机磁盘。read_image_for_vision 负责先解析工作区路径、检查图片格式和尺寸；后续逻辑再把图像内容组织成多模态消息，模型才有机会看到图像。

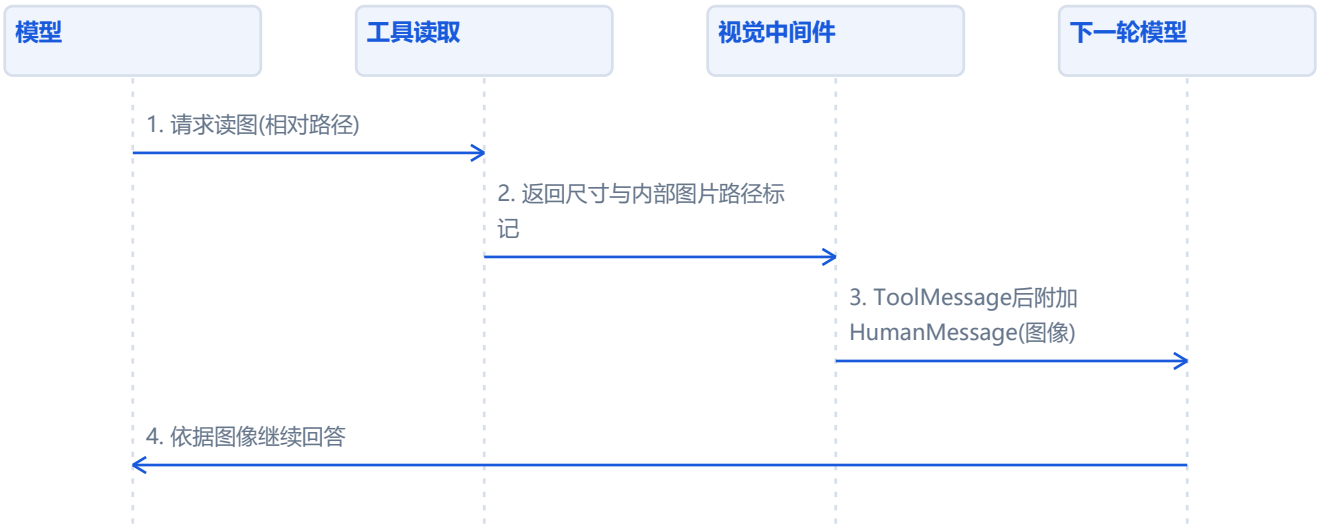


图7-3 实际主动附图路径（省略传输细节）

数据	作用	不能误认为
相对路径	表达工作区内目标	图像像素本身
工具结果JSON	报告读取元信息	必然已发送附图
image_url内容块	实际提供图像输入	本地检测器的bbox结果

源码定位：workspace_file_reader.py::read_image_for_vision_tool 与 vision.py::VisionToolResultMiddleware。读取失败时不能仍然假设模型看过图；服务不支持视觉时也不应强行附图。

7.7 视觉fallback：首轮漏调工具才兜底

Fallback（回退/兜底）在本项目里指一种特定策略：第一轮模型没有提出工具调用，且满足视觉条件时，补一次读图过程。它不是“主模型坏了自动换备用模型”，也不是每轮回答都先读图。



图7-4 实际视觉兜底判断（其他条件省略于图，正文补充）

源码节选：LR-Agent/vendor/local-agent/app/agent/langchain_runtime/vision.py::VisionFallbackMiddleware._compute（轮次守卫段）

```
rounds = (
    int(state.get("vision_rounds") or 0) + 1
    if isinstance(state, dict) else 1
)
update: dict[str, Any] = {"vision_rounds": rounds}
if rounds > 1:
    return update
```

isinstance 判断对象类型；dict[str, Any]只是类型标注。每次进入都更新轮次，即使本轮不读图也要记录，避免第二轮被误当首轮。后续判断和消息注入未显示在此节选中。兜底调用仍占用模型预算，不能当作免费额外一轮。

7.8 本地inference：另一套输入输出协议

Inference指已训练模型对新样本做预测。当前推理服务是独立Python进程，使用JSON Lines：一行一个JSON请求，一行一个JSON响应。它与面向外部调用方的SSE不是同一种协议；不要把两个接口的字段混着发送。

字段/命令	含义	实例或约束
cmd=run	运行一次推理	request为实际推理参数
request.kind	选择算法入口	yolo_detect等
imagePath	输入图片路径	由执行方提供可访问路径
model.checkpointPath	权重位置	缺失时抛ValueError

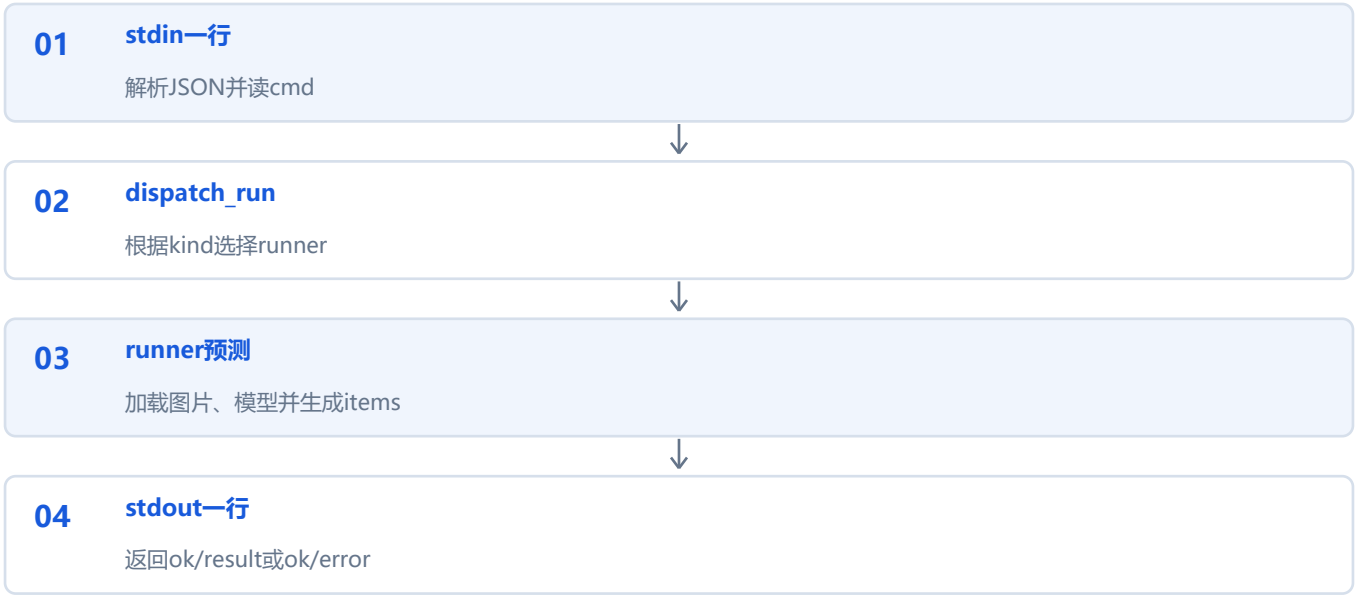


图7-5 实际本地推理服务入口

函数与错误

server.py::_handle_run 返回一个字典；dispatch_run 的不支持类型会抛异常。异常在服务层变成ok=false，并可能带trace。教学日志不要公开trace中的个人路径或请求凭据。

7.9 bbox名词：先明确坐标约定

Bounding box（边界框）通常有两种表示：左上角加宽高，或两个角点。YOLO runner 的普通矩形结果使用归一化的 x、y、width、height，其中x/y表示左上角，不能误作中心。归一化指除以整图宽高，便于不同尺寸图片共用表示。

ROI是Region of Interest（感兴趣区域），本例指裁切区域。cx/cy是裁切图内的中心像素坐标，bw/bh是框宽高；off_x/off_y是裁切原点相对于整图的偏移。先转左上角，再加偏移，最后归一化。

项目结果字段	含义	1000×800示例
x / y	左上角相对位置	0.15 / 0.325 → 150/260像素
width / height	框宽高相对尺寸	0.2 / 0.1 → 200/80像素
classId / className	模型类别编号和名称	0 / label
confidence	模型输出置信分数	0.9不是准确率90%

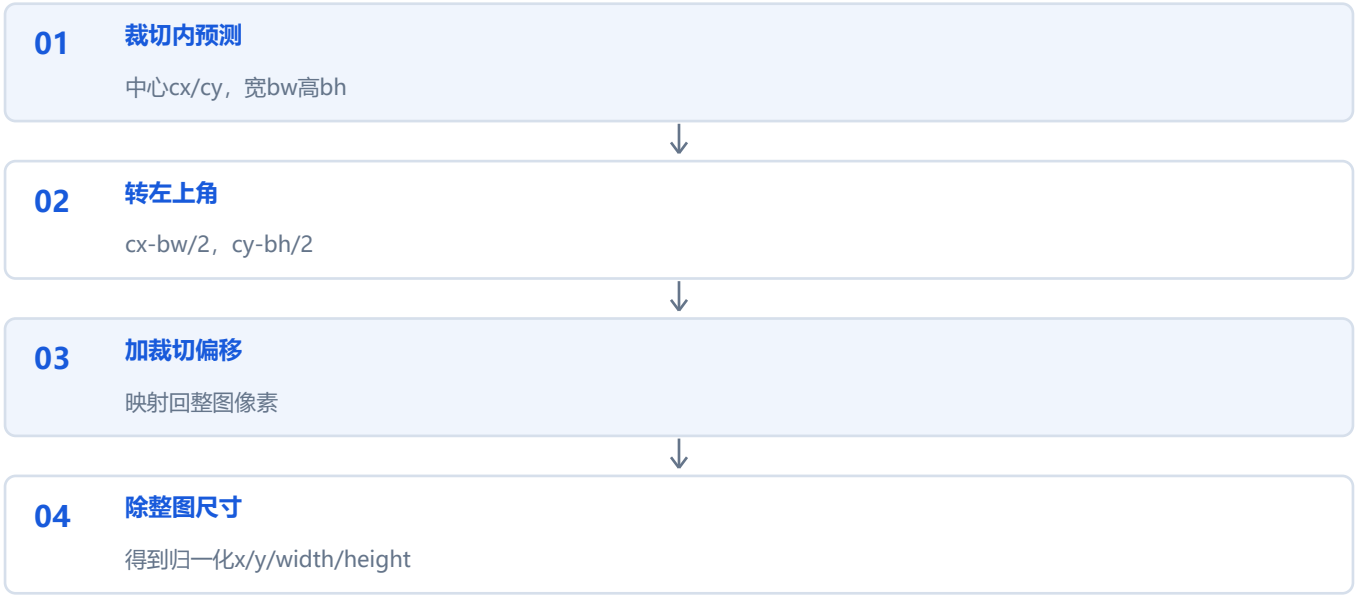


图7-6 概念坐标转换：含ROI偏移

不是所有几何都相同

旋转框OBB使用中心、宽高、角度；SAM2与关键点也有不同几何。当前章节仅用普通bbox演示，不能把这一套公式直接套到所有标注类型。

7.10 完整实验：手算并校验一个bbox

用纯数字复现坐标转换，可先排除图片加载、GPU和权重环境的干扰。实验验证公式与边界，不运行目标检测。请先在纸上算出左上角(150,260)，再运行代码核对。

完整教学实验 ch07_bbox。在 study 自行新建 my_ch07_bbox.py，复制全部代码。

```python
import json

def normalized_box(cx, cy, bw, bh, off_x, off_y, width, height):
    return {
        'x': (off_x + cx - bw / 2) / width,
        'y': (off_y + cy - bh / 2) / height,
        'width': bw / width,
        'height': bh / height,
    }

def is_inside(box):
    return (0 <= box['x'] < 1 and 0 <= box['y'] < 1
            and box['width'] > 0 and box['height'] > 0
            and box['x'] + box['width'] <= 1
            and box['y'] + box['height'] <= 1)

box = normalized_box(150, 100, 200, 80, 100, 200, 1000, 800)
assert box == {'x': 0.15, 'y': 0.325, 'width': 0.2, 'height': 0.1}
assert is_inside(box)
overflow = dict(box, x=0.95)
assert not is_inside(overflow)
print(json.dumps(box, sort_keys=True))
print('pixel_left_top:', round(box['x'] * 1000),
      round(box['y'] * 800))
print('overflow_rejected:', not is_inside(overflow))
```

运行：& $agentPy ".\study\my_ch07_bbox.py"（工作区根目录）

本轮实际输出：

```
{"height": 0.1, "width": 0.2, "x": 0.15, "y": 0.325}
pixel_left_top: 150 260
overflow_rejected: True
```

normalized_box 是教学函数，八个参数依次表示裁切图中的中心、框宽高、裁切原点偏移、整图宽高；返回归一化字典。

整图1000×800，裁切原点(100,200)，预测中心(150,100)、宽200高80。先减半宽半高得到裁切图左上角，再加偏移，最后除整图尺寸。

项目源码对各字段分别夹紧到0至1；本实验额外检查右边界与下边界。它是边界学习示例，不宣称生产所有输出已经经过这个额外函数。

一个重要边界

每个值处于0到1之间，不等于整个框都在图内。x=0.95、width=0.2会超出右边界；要检查它们的和。还要单独考虑NaN、零尺寸和类别映射等生产约束。

7.11 从推理结果到提案：完成不等于保存

检测器返回items后，还需要将模型类别映射到项目标签、构造待确认变更并回传执行结果。本项目将自动标注作为外部执行工具：后端发tool_pending，执行方完成后按同一tool_call_id回传，后端继续生成说明。不要因为status为completed就声称已经写入磁盘。

结果层级	回答的问题	应观察的证据
推理完成	是否获得预测items	ok、items、类别和geometry
提案待确认	是否需要用户决定变更	proposal_pending=True
文件已写入	确认后的结果是否保存	file_written与实际文件回读
重启后回读	保存是否仍可读且一致	重启前后结构比对



图7-7 后端视角的实际协议关系

标签也要对齐

模型类别编号不自动等于项目标签ID。讲项目时应说明映射和未知类别如何处理，并用实际样本验证；本章不调用任何实际写盘工具。

7.12 完整实验：验证结果状态的独立含义

下面使用真实的工具结果封装函数，将一个固定推理结果表达为“已产生提案但尚未写盘”。重点是字段语义；不是完整的auto_annotate实现。把结构化状态与给人看的summary分开，可以避免仅靠自然语言判断是否保存。

完整教学实验 ch07_result。在 study 自行新建 my_ch07_result.py，复制全部代码。

```python
import json
from app.agent.tools.tool_result import build_tool_result, parse_tool_result

inference = {
    'items': [{'className': 'label', 'classId': 0, 'confidence': 0.9,
               'geometry': {'x': 0.1, 'y': 0.2,
                            'width': 0.3, 'height': 0.4}}],
    'classNames': ['label'],
}
text = build_tool_result(
    ok=True, tool='auto_annotate', status='completed',
    summary='created 1 proposal', proposal_pending=True,
    item_count=len(inference['items']),
)
payload = parse_tool_result(text)
assert payload is not None
assert payload['file_written'] is False
assert payload['proposal_pending'] is True
print('status:', payload['status'])
print('proposal_pending:', payload['proposal_pending'])
print('file_written:', payload['file_written'])
print('items:', payload['item_count'])
```

运行：& $agentPy ".\study\my_ch07_result.py"（工作区根目录）

本轮实际输出：

```
status: completed
proposal_pending: True
file_written: False
items: 1
```

此处 inference 是固定的教学输入，没有运行YOLO。len 取实例数；build_tool_result 返回JSON字符串，parse_tool_result 将字符串解析为字典或 None。

status=completed 描述工具执行完成；proposal_pending=True 描述仍有待确认变更；file_written=False 描述未写盘。三个字段回答不同问题，不能互相替代。

没有打开文件、没有调用落盘工具，也没有真实提案应用。本实验只验证项目结果封装的默认值与语义。

返回值为什么是字符串

ToolMessage.content 常需要字符串，因此结果先序列化成JSON文本。解析后才能按布尔字段判断，不应使用“摘要中是否包含保存”作为业务规则。

7.13 真实验收图：把截图当作证据的一部分

已有验收日期为2026-10-17：两张副本图片通过真实语言模型工具调用与本地YOLO，先产生待确认结果，确认后写入，再完整重启回读；两份标注文档与重启前结构相同。此页引用已有记录，本章没有再次调用云模型或推理权重。



图7-8 已有真实项目截图：确认保存并重启后重新打开的矩形框与标签；2026-10-17验收，两张图片范围。

看图时依次问：图像上有框，只能证明界面显示；列表有标签，说明实例已进入当前文档；持久化还需结合磁盘文件、正式回读和重启一致性。后端学习关注这些证据之间如何闭环，无须学习桌面页面实现。

截图不能替代指标

一张看起来正确的框无法证明整体精度、召回率或人工效率。原始数据和日志保存在本地忽略目录；教材不展示账号、模型Key或登录会话。

7.14 功能正确、模型正确、工作更快：分开验证

接入工具后最容易过度推断：“测试通过，所以模型准确”“两张图成功，所以效率提升”。功能验收、检测质量和工作效率属于不同问题，应分别准备输入、判定规则和证据。它们可以互相补充，但不能互相冒充。

问题	怎样验证	不能从中推出
协议是否跑通	固定模型/工具+事件字段断言	模型理解能力达标
检测是否可靠	独立标注集+类别/IoU匹配	用户实际省时
人工是否省时	固定任务对照+计时与返工记录	跨数据集同等效果
保存是否可靠	确认前后+重启回读一致	所有并发场景安全

置信度阈值控制保留哪些预测，调高通常更保守，但实际精确率和召回率变化要用本项目数据测。源码默认阈值与已有验收使用的0.3不是同一个事实：前者是未传参数时的代码行为，后者是特定运行配置。

面试表达

可以说“建立了从事件到回读的功能验收思路”；若尚未亲测，只能说“阅读并理解已有验收”。没有评估集就不写准确率提升。

7.15 练习：换模型、换图片时先查什么

练习A：辅助模型配置缺一个字段会怎样?

参考：build_llm_pair要求aux_model、aux_api_key、aux_base_url三者非空才构造。否则aux_llm为None，子代理装配处的aux_llm or llm回退主模型。测试可复制工厂实验，只把其中一个字段改为空，并断言工厂只被调用一次。

练习B：为什么首轮真实工具调用后不再强行读图?

参考：这是首轮漏调视觉工具的兜底，不是每轮强制读图。vision_rounds无条件增长能防止后续回合被误判为首轮；若模型主动读图，另一个中间件负责工具结果后的附图。

练习C：置信度0.9是否说明准确率90%?

参考：不说明。它是模型对当前预测的分数；准确率、精确率或召回率是基于数据集与匹配规则计算的评价。应先定义评估任务，再决定指标。

你要提交的证据	最低内容	合格标准
工厂实验记录	配置占位值、调用次数和输出	无真实网络请求
坐标手算记录	像素、偏移、归一化公式	能解释溢出框
状态说明	completed/pending/written区别	不把待确认写成已保存

进阶但不抢跑

确认本章概念后，再独立检查未知类别与越界框的真实策略；先给失败输入，再决定是否改代码，不为简历凭空扩展模型功能。

7.16 面试复盘：接入能力的边界要说清

30秒概念答法

“Agent用语言模型决定何时调用工具，检测模型通过工具执行图像预测。我们把二者的输入输出协议分开：主模型接消息，推理侧接图片和权重参数，返回类别、置信度与几何，再进入待确认业务流程。”

2分钟项目答法

先用一句自动标注需求开场，按主模型构造、工具调用、外部推理、结果回传和确认保存解释链路。指出辅助模型仅用于子代理，视觉fallback也不是自动换模型。再展示一个你亲自运行的配置或坐标实验，最后说明当前只掌握了什么、尚未评估什么。

面试追问	回答思路	个人证据
为何分离Agent与推理环境	依赖和生命周期不同，便于隔离	解释器/协议定位
bbox画偏了怎样查	先查坐标约定、ROI偏移和尺寸	手算与离线断言
为什么不声称精度提升	功能合同没有测检测质量	独立评估计划
工具完成是否已保存	区分执行、提案与写盘	结果字段和回读证据

写入学习日志

分别记“阅读理解” “本人复现” “本人改造”。本章三个实验若由你重跑，可记录复现；已有源码与旧验收不能包装为你从零实现或本人跑过的所有场景。

7.17 图解：一行JSON怎样表达一次推理

下面仅为结构阅读示例，不执行。imagePath和checkpointPath是示例绝对路径；params实际位于model内部。每个JSON对象作为一整行发送，显示换行仅为教学排版。

教学简化示例：请求结构：LR-Agent/vendor/inference/server.py::_handle_run

```
{
  "cmd": "run",
  "request": {
    "kind": "yolo_detect",
    "imagePath": "D:/example/a.jpg",
    "model": {
      "checkpointPath": "D:/example/model.pt",
      "params": {
        "confThreshold": 0.7
      }
    }
  }
}
```

教学简化示例：成功响应形状：server.py::_handle_run

```
{"ok": true, "result": {"items": []}}
```

result的具体内容由runner定义；示例空items不表示已经完成真实检测。失败由服务层返回ok=false及error等字段；该JSON Lines协议与SSE的data帧不同。

7.18 源码图解：模型配置怎样进入实例

先看图中的真实路径与原始行号，再打开源文件跟读。签名说明输入条件，函数体说明处理，return说明交给下一层的数据。图为源码阅读辅助，不替代完整上下文。

真实源码阅读图 / build_llm_pair

LR-Agent/vendor/local-agent/app/agent/langchain_runtime/entry.py

```
190 def build_llm_pair(body: LocalChatStreamRequest) -> tuple[Any, Any]:
... # 文档说明省略，以下保留原始行号
197     global ChatOpenAI  # noqa: PLW0603 - 惰性缓存，避免 baseline 环境 import langchain_openai
198     if ChatOpenAI is None:
199         ChatOpenAI = _load_chat_openai()
200     factory = ChatOpenAI
201
202     base_url = body.base_url.rstrip("/")
203     llm = factory(
204         model=body.model,
205         api_key=body.api_key,
206         base_url=base_url,
207         streaming=True,
208         temperature=0.7,
209         timeout=120,
210         extra_body=_deepseek_extra_body(body.model, base_url),
211     )
212
213     aux_llm: Any = None
214     if body.aux_model.strip() and body.aux_api_key.strip() and body.aux_base_url.strip():
215         aux_base_url = body.aux_base_url.strip().rstrip("/")
216         aux_llm = factory(
217             model=body.aux_model.strip(),
218             api_key=body.aux_api_key.strip(),
219             base_url=aux_base_url,
220             streaming=True,
221             temperature=0.7,
222             timeout=120,
223             extra_body=_deepseek_extra_body(body.aux_model, aux_base_url),
```

真实源码阅读图：LR-Agent/vendor/local-agent/app/agent/langchain_runtime/entry.py::build_llm_pair。按当前文件原始行号截取并排版生成。

先看什么	怎样读	自己记录什么
输入	区分实参、配置与上下文	哪些字段可缺失
处理	找条件、调用和错误分支	哪一步改变消息或执行
输出	确认返回值与接收者	怎样用实验检查

源码与证据索引

LR-Agent/vendor/local-agent/app/agent/langchain_runtime/entry.py::build_llm_pair/build_engine_agent

LR-Agent/vendor/local-agent/app/agent/langchain_runtime/vision.py::VisionFallbackMiddleware/VisionToolResultMiddleware

LR-Agent/vendor/local-agent/app/agent/tools/workspace_file_reader.py::read_image_for_vision_tool

LR-Agent/vendor/local-agent/app/agent/tools/tool_result.py::build_tool_result/parse_tool_result

LR-Agent/vendor/inference/server.py::_handle_run/main

LR-Agent/vendor/inference/runners/dispatch.py::dispatch_run

LR-Agent/vendor/inference/runners/yolo.py::_run_yolo_predict

docs/2026-10-17本地主流程验收记录.md::真实模型、推理和确认落盘

阅读时先定位符号，再看完整输入、处理与返回。源码存在、离线实验通过和已有真实验收属于不同证据；未验证的模型质量、效率与并发结论仍需单独实验。